

=====

HW2: Traditional Software Design vs Function-as-a-Service

Hospital Management System

Course: HW/SW Co-Design (00460882)

=====

PART 1 — TRADITIONAL SYSTEM IMPLEMENTATION

System Chosen: Hospital Management System

We implemented a hospital management system in C++17 using a traditional monolithic architecture. The system is organized as a single HospitalSystem class that owns all shared state and exposes 8 core operations.

Data Model

The system maintains the following entities in shared memory:

- Patient : id, name, bed assignment, department, admission status, bill
- Doctor : id, name, department, shift (MORNING/EVENING/NIGHT/OFF), availability
- Nurse : id, department assignment, availability
- OperatingRoom : id, availability, assigned doctor/patient, procedure name
- Equipment : id, type (ventilator/monitor), allocation status
- Bed : id, department, occupancy, patient assignment
- EREntry : patient id, severity (LOW/MEDIUM/HIGH/CRITICAL), symptoms

The 8 Operations

1. admitPatient(id, name, dept)

Finds a free bed in the requested department, creates a patient record, and marks the bed as occupied.

2. dischargePatient(id)

Frees the patient's bed, releases any allocated equipment, marks patient as discharged.

3. assignDoctorShift(doctor_id, shift)

Updates the doctor's shift and availability flag.

4. assignNurseToDept(nurse_id, dept_id)

Reassigns a nurse to a department.

5. scheduleOR(or_id, doctor_id, patient_id, procedure)

Books an operating room, marks both the room and doctor as unavailable.

6. handleERAdmission(patient_id, severity, symptoms)

Adds a patient to the ER queue and re-sorts by severity (CRITICAL first).

7. allocateEquipment(equip_id, dept_id, patient_id)

Assigns a piece of equipment to a patient in a department.

8. generateBill(patient_id)

Computes total bill: base charge + equipment fees + doctor fee.

Architecture

All state lives inside a single HospitalSystem object. Operations call each other's helper methods directly (e.g., admitPatient calls findFreeBed which scans the shared beds_vector). All data is in-memory and accessed directly

by reference — no copying, no serialization, no dispatch overhead.

Files:

Traditional/hospital.h — class and struct definitions

Traditional/hospital.cpp — all 8 operation implementations

Traditional/main.cpp — workload runner (5000 iterations, 8 ops each)

PART 2 — FUNCTION-AS-A-SERVICE (FaaS) IMPLEMENTATION

The same 8 operations are re-implemented as independent, stateless functions.

No function holds any memory between invocations. Each function receives a complete event struct describing its trigger, and a Database reference that simulates a cloud state store (e.g., DynamoDB in a real FaaS deployment).

FaaS Principles Applied

1. Stateless execution

Each function (fn_admit_patient, fn_discharge_patient, etc.) receives ALL the information it needs via an event struct. Between invocations, no state is held inside the function itself. The Database object is the external state store — analogous to a cloud KV store.

2. Isolated functionality

Each function lives in its own .cpp file and has no knowledge of the others. fn_schedule_or.cpp does not import or call fn_assign_nurse.cpp. A compilation failure or runtime error in one function does not prevent

the others from being built or invoked.

3. Minimal dependencies between functions

Functions communicate only through the shared Database, not through direct calls. If a feature requires coordination between two operations (e.g., mass casualty reallocation), it is implemented as a new independent function that reads and writes the Database.

The Dispatcher

A Dispatcher class maintains a `std::unordered_map<string, Handler>` where `Handler` is a `std::function<std::any(const std::any&, Database&)>`. Every invocation pays the cost of:

- A hash-map name lookup
- `std::any` boxing (wrapping the event struct in a type-erased container)
- Lambda call through `std::function`
- `std::any` unboxing (casting back to the concrete event type)

This indirection is deliberate — it mirrors how a real FaaS runtime (AWS Lambda, Google Cloud Functions) routes HTTP triggers or event queue messages to the correct handler, adding per-invocation overhead that is invisible in a traditional call.

Files:

- FaaS/db.h — Database struct (the state store)
- FaaS/events.h — event and result structs for all functions
- FaaS/dispatcher.h / .cpp — runtime router
- FaaS/fn_admit_patient.h/.cpp
- FaaS/fn_discharge_patient.h/.cpp
- FaaS/fn_assign_doctor_shift.h/.cpp
- FaaS/fn_assign_nurse.h/.cpp

FaaS/fn_schedule_or.h/.cpp

FaaS/fn_er_admission.h/.cpp

FaaS/fn_allocate_equipment.h/.cpp

FaaS/fn_generate_bill.h/.cpp

FaaS/main.cpp — workload runner (same 5000 iterations)

PART 3 — FEATURE EXTENSION CHALLENGE

New Feature: Mass Casualty Event Protocol

When the ER queue exceeds 80% of its capacity, the system automatically:

1. Reallocates nurses from non-critical departments (ortho, derm) to the ER
2. Cancels all elective OR bookings and frees those rooms and doctors
3. Counts available doctors to alert for ER duty

This is a realistic and complex feature because it crosses three subsystems simultaneously: nurse assignments, OR scheduling, and doctor availability.

Traditional Architecture — What Had to Change

Files modified: 2 (hospital.h, hospital.cpp)

New files: 0

In hospital.h we added:

- A new result struct `MassCasualtyResult`
- A new method declaration `handleMassCasualty(int er_capacity, float threshold)`

In `hospital.cpp` we added a 40-line method that directly accesses three private member vectors: `nurses_`, `or_rooms_`, and `doctors_` — all in the same function body.

Risk assessment:

HIGH. The method mutates shared state across three subsystems in a single call. If the loop over `or_rooms_` contains a bug that writes a wrong `doctor_id`, it silently corrupts the `doctors_` vector, which then breaks every subsequent call to `scheduleOR()` and `assignDoctorShift()` for the rest of the program run. There is no isolation boundary — one bug propagates system-wide immediately.

Additionally, adding a method to `HospitalSystem` requires recompiling the entire system (`hospital.cpp` is a translation unit shared by all features).

In a large production codebase this means rebuilding and retesting every feature that depends on the class.

FaaS Architecture — What Had to Change

Files modified: 2 (`events.h` — 2 new structs, `dispatcher.cpp` — 1 new line)

New files: 2 (`fn_mass_casualty.h`, `fn_mass_casualty.cpp`)

The 8 existing function files were not touched at all.

In `events.h` we added:

```
struct MassCasualtyEvent { int er_capacity; float threshold; };
```

```
struct MassCasualtyResult { int nurses_reallocated; int or_bookings_cancelled;
                           int doctors_alerted;  bool triggered; };
```

In dispatcher.cpp we added one registration:

```
d.register_function("mass_casualty", [](const std::any& ev, Database& db)
    -> std::any { return fn_mass_casualty(std::any_cast<MassCasualtyEvent>(ev), db); });
```

fn_mass_casualty.cpp contains the full logic — identical logic to the Traditional version — but as a standalone file.

Risk assessment:

LOW. If fn_mass_casualty.cpp contains a bug, the worst case is that this one function returns an incorrect result or crashes on invocation. The other 8 functions — fn_admit_patient, fn_schedule_or, etc. — are completely unaffected. They were not recompiled, not modified, and share no code path with the new function. In a real cloud deployment, a crashing Lambda function is restarted in isolation; it does not take down the rest of the service.

Comparison Summary

	Traditional	FaaS
Files changed	2	2
New files	0	2
Existing functions	all recompiled	none recompiled
Risk of regression	High (shared state)	Low (isolated)
Failure blast radius	Entire system	One function only
Deployment unit	Whole binary	Single function

PART 4 — PERFORMANCE EVALUATION

Setup

Both binaries compiled with: `g++ -O0 -g -fno-omit-frame-pointer -std=c++17`

Workload: 5000 iterations × (8 hospital operations + mass casualty check)

= 1,550,000 total operations per run

Platform: Ubuntu 22.04, QEMU/KVM VM, kernel 5.15.0-1080-kvm

Tool: perf stat with events:

cycles, instructions, cache-misses, cache-references,

L1-dcache-load-misses, context-switches, task-clock

Profiling command:

```
perf record -e cpu-clock --call-graph fp -F 99 -o perf.data ./binary
```

```
perf script | stackcollapse-perf.pl | flamegraph.pl > flame.svg
```

Results Table

Metric	Traditional	FaaS	Ratio
Elapsed time	1.979 s	2.565 s	+30%
task-clock (CPU time)	1971 ms	2561 ms	+30%
CPU Cycles	4,685,871,661	6,077,662,984	+30%
Instructions retired	9,804,808,408	12,236,553,367	+25%

IPC (insn per cycle)	2.09	2.01	-4%
Cache references	642,445	1,175,670	+83%
Cache misses	4,091 (0.64%)	5,392 (0.46%)	+32%
L1-dcache-load-misses	3,807,415	2,914,898	-24%
Context switches	17	20	~same

Analysis

1. Traditional is 30% faster (1.979s vs 2.565s)

The Traditional binary completes the same 1.55M operations in 30% less time and 30% fewer CPU cycles. Both binaries run at the same clock frequency (2.37–2.38 GHz), confirming the difference is in work done, not in CPU speed variation.

2. FaaS executes 25% more instructions for the same logical work

Traditional: 9.80 billion instructions

FaaS: 12.24 billion instructions

The extra 2.44 billion instructions come entirely from the dispatch layer: `unordered_map::find()`, `std::any` construction and destruction, `std::function` call through a lambda, and `any_cast<T>` unwrapping. Just as spawning a new thread per task wastes cycles on thread creation and context management, the FaaS dispatcher wastes cycles on indirection.

3. FaaS accesses 83% more cache lines

Cache references: 1,175,670 (FaaS) vs 642,445 (Traditional).

The dispatcher's hash map, the `std::any` type-erased storage, and the lambda capture list all require additional heap allocations and pointer dereferences that touch new cache lines on every invocation.

4. IPC is slightly lower for FaaS (2.01 vs 2.09)

Fewer instructions are completed per cycle, indicating that the dispatcher's pointer-heavy code causes more pipeline stalls than the Traditional's direct vector iterations.

5. L1 cache misses are lower in FaaS (2.91M vs 3.81M)

This is a counter-intuitive result. The Traditional system iterates over large vectors (`patients_`, `beds_`, `equipment_`) in hot paths like `dischargePatient()` and `generateBill()`. These sequential scans generate many L1 misses when the working set exceeds the L1 cache. FaaS functions return early more often (e.g., `fn_schedule_or` returns immediately if the OR is unavailable) and access smaller slices of the database per call, resulting in fewer total L1 misses — but this does not compensate for the higher overall instruction count.

6. Context switches are essentially the same (17 vs 20)

Both implementations are single-threaded. The small difference is OS scheduling noise, not an architectural property.

Flame Graph Analysis

Traditional flame graph (`flame_traditional.svg`):

The graph shows a narrow, tall stack rooted in `main()` calling directly into `HospitalSystem` methods (`admitPatient`, `scheduleOR`, `generateBill` etc.).

The hot frames are the vector scan loops inside the helper methods (`findPatient`, `findEquipment`, `findFreeBed`). The call chain is short and direct — no indirection layer is visible.

FaaS flame graph (flame_faas.svg):

The graph shows a wider base dominated by `Dispatcher::invoke()` and the `std::any` machinery (`any_cast`, `__any_dispatch`). Below the dispatcher, each actual function (`fn_admit_patient`, `fn_schedule_or`, etc.) appears as a separate stack column of similar height. The dispatcher overhead is clearly visible as a distinct layer between `main()` and the actual work, mirroring how the tutorial's "New Thread per Task" flame graph showed thread-management overhead as a wide base layer beneath the actual task work.

Conclusion

The Traditional architecture is the better performer in this benchmark: 30% faster wall time, 25% fewer instructions, and lower dispatch overhead. This is expected: when all operations run in a single process with shared memory and direct function calls, there is no management layer consuming cycles.

The FaaS overhead is real and measurable, but it comes with architectural benefits (Part 3): isolated failure domains, independent deployability, and additive extensibility. Whether the trade-off is worthwhile depends on the use case — for high-throughput in-process computation, Traditional wins; for independently scalable, fault-isolated services, FaaS wins.

=====

AI Tool Usage Disclosure

=====

Claude Code (Anthropic) was used as an AI assistant in this assignment for:

- Proposing the Mass Casualty Event Protocol feature (Part 3)
- Generating the initial C++ code structure for both architectures
- Suggesting the perf profiling commands and flamegraph pipeline

All architectural decisions, analysis, and interpretation of perf results were reviewed and verified by the students. The code was compiled and run on the course VM (naranja14) and all numbers in this report are from actual perf stat output.

=====